
Collections in Rust

`(a, b)`

`[T; N]`

`Vec<T>`

`HashSet<T>`

`HashMap<K, V>`



Gábor Takács

`<dr.gabortakacs@gmail.com>`

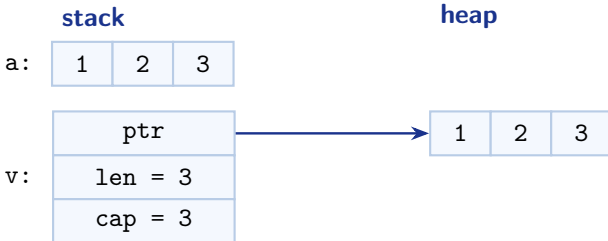
From Python to Rust

Python	Rust	Rust type	Size
(1, 2.5)	(1, 2.5)	(i32, f64)	fixed
—	[1, 2, 3]	[i32; 3]	fixed
[1, 2, 3]	vec![1, 2, 3]	Vec<i32>	growable
{1, 2}	HashSet::from([1, 2])	HashSet<i32>	growable
{1: 2.5}	HashMap::from([(1, 2.5)])	HashMap<i32, f64>	growable

- ▶ Mutability belongs to the **variable** (`let mut`), not to the type: a `mut` tuple can be modified too.
- ▶ The real split in Rust is **fixed size** vs. **growable**.
- ▶ Rust's array is a list of fixed length, whose size is part of its type.

Where does the data live?

```
let a = [1, 2, 3];      // all 3 elements on the stack
let v = vec![1, 2, 3];  // ptr, len and cap on the stack,
                        // the 3 elements on the heap
```



- ▶ A tuple and an array are stored **in place**, just like an integer: their size is known at compile time, so they need no heap.
- ▶ A `Vec` is laid out exactly like a `String`. This is no accident: a `String` is a `Vec<u8>` whose bytes are guaranteed to be valid UTF-8.

The element types are part of the type

```
let v = vec![1, 2, 3];    // Vec<i32>
let w = vec![1, 2.5];    // error[E0308]: mismatched types
let t = (1, 2.5, 'x');    // (i32, f64, char): works fine
```

```
let mut e = Vec::new();  // element type unknown yet,
e.push(4.5);             // from here on: Vec<f64>

let e = Vec::new();      // error[E0282]:
                        // type annotations needed
let e = Vec::<f64>::new(); // works fine
```

- ▶ An array, a `Vec`, a `HashSet` and a `HashMap` all have **one element type**.
- ▶ Only a tuple can mix types: its length is fixed, so every position can have a type of its own.
- ▶ The element type is **inferred**, like any other type.

Indexing: `[i]` or `get(i)`

```
let v = vec![10, 20, 30];  
println!("{}", v[1]); // 20  
println!("{}", v[5]); // panic: index out of bounds:  
                        // the len is 3 but the index is 5
```

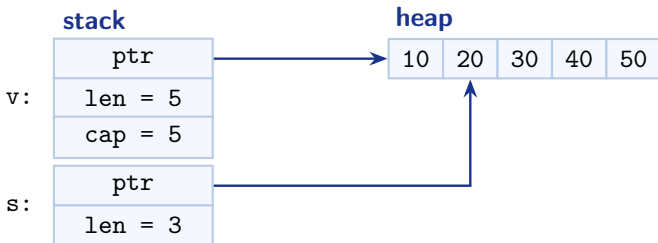
```
match v.get(5) {           // Some(&element) or None  
    Some(x) => println!("{}", x),  
    None => println!("no such element"),  
}
```

- ▶ Every index is **checked** at runtime:[†] reading past the end, a classic bug in C, is impossible. `v[i]` stops the program with a **panic**.
- ▶ `v.get(i)` returns an **Option** instead: **Some**(&element) or **None**.
- ▶ An index must be a **usize**: indexing with an **i32** gives error[E0277].

[†]An array indexed by a constant is checked already at compile time.

A slice: a borrowed view of elements

```
let v = vec![10, 20, 30, 40, 50];  
let s = &v[1..4]; // [20, 30, 40], its type is &[i32]
```



- ▶ A slice is a **pointer and a length**: it borrows, it owns nothing.[†]
- ▶ It works the same way on an array: `&a[..2]`.
- ▶ While `s` lives, `v` cannot be modified: a `push` might move the buffer.

[†]A `&str` is the same for text: a slice of UTF-8 bytes.

4 types in principle, 3 in practice, 2 in use

Type	What it is	Use it?	For text
<code>Vec<T></code>	owned, growable elements (heap)	yes: to own or build	<code>String</code>
<code>&Vec<T></code>	a borrow of a <code>Vec</code>	no: take a <code>&[T]</code>	<code>&String</code>
<code>[T]</code>	the elements themselves	cannot: size unknown	<code>str</code>
<code>&[T]</code>	a borrow of any run of elements	yes: to read	<code>&str</code>

```
let v = vec![1, 2, 3];
let a = [4, 5, 6];
let s: &[i32] = &v; // a &Vec<i32> becomes a &[i32]
let t: &[i32] = &a; // and so does a &[i32; 3]
```

- ▶ `[T]` has no fixed size, so it cannot be put into a variable on its own.
- ▶ Wherever a `&[T]` is expected, a `&Vec<T>` is turned into one automatically (**deref coercion**), and so is a reference to an array.

A collection owns its elements

```
let s = "hello".to_owned();
let mut v = Vec::new();
v.push(s);                // s is moved into v
println!("{s}");          // error[E0382]: borrow of moved value: `s`
```

```
let t = v[0];              // error[E0507]: cannot move out of index
let r = &v[0];             // borrow it
let c = v[0].clone();      // copy it
let o = v.remove(0);       // take it out, v gets shorter
```

- ▶ When `v` is dropped, all of its elements are dropped with it.
- ▶ Moving an element out by indexing would leave a **hole** in `v`, so it is not allowed: borrow it, clone it, or remove it.
- ▶ Elements of a **Copy** type, like `i32`, are simply copied by `v[0]`.

Three ways to loop over a collection

```
let mut v = vec![1, 2, 3];  
for x in &v { println!("{x}"); }    // reads the elements  
for x in &mut v { *x *= 10; }        // modifies them in place  
for x in v { println!("{x}"); }     // takes them over  
println!("{v:?}"); // error[E0382]: borrow of moved value: `v`
```

Loop	x is	After the loop, v is
for x in &v	&i32	unchanged
for x in &mut v	&mut i32	modified
for x in v	i32	consumed by the loop

- ▶ These are exactly the three ways of passing a value: shared borrow, mutable borrow, and move.
- ▶ `*x` is the element that the reference `x` points to.

No modification while looping

```
let mut v = vec![1, 2, 3];
for x in &v {           // v is borrowed for the whole loop
    v.push(x * 10);      // error[E0502]: cannot borrow `v` as
}                        // mutable because it is also
                        // borrowed as immutable
```

```
let mut extra = Vec::new();
for x in &v {
    extra.push(x * 10);  // collect the changes first,
}
v.extend(extra);        // then apply them: [1, 2, 3, 10, 20, 30]
```

- ▶ A push might move the buffer, and `x` would point to freed memory.[†]
- ▶ In many languages a runtime bug, in Rust a **compile-time error**.

[†]With `for x in &mut v`, the same push gives error [E0499].

HashMap: values looked up by keys

```
use std::collections::HashMap;

let mut ages = HashMap::new();
ages.insert("Alice".to_owned(), 30); // key and value
ages.insert("Bob".to_owned(), 25);   // are moved in
ages.insert("Alice".to_owned(), 31); // overwrites 30
println!("{:?}", ages.get("Alice")); // Some(31)
println!("{:?}", ages.get("Carol")); // None
```

```
let mut counts = HashMap::new();
for c in "banana".chars() {
    *counts.entry(c).or_insert(0) += 1;
}
println!("{counts:?}"); // {'n': 2, 'b': 1, 'a': 3}
```

- ▶ `entry(c).or_insert(0)`: the value of `c`, inserting a 0 first if it is missing.
- ▶ The order of the pairs is **unspecified**: it may even differ from run to run.

HashSet: each element at most once

```
use std::collections::HashSet;

let mut s = HashSet::new();
println!("{}", s.insert(1));    // true: it was new
println!("{}", s.insert(1));    // false: it was there already
println!("{}", s.contains(&1)); // true
```

```
let a = HashSet::from([1, 2, 3]);
let b = HashSet::from([2, 3, 4]);
println!("{}", &a & &b); // {2, 3}           intersection
println!("{}", &a | &b); // {1, 2, 3, 4}      union
println!("{}", &a - &b); // {1}              difference
```

- ▶ The operators borrow both sides, so `a` and `b` stay usable: the result is a new set.
- ▶ The order of the elements is **unspecified**, as in a `HashMap`.[†]

[†]In fact, a `HashSet<T>` is built on a `HashMap<T, ()>`: keys without values.

Converting between collections: collect()

```
let a = [3, 1, 3, 2];  
let v = Vec::from(a);           // [3, 1, 3, 2]  
let s: HashSet<i32> = v.into_iter().collect(); // {1, 2, 3}  
let w: Vec<i32> = s.into_iter().collect();    // e.g. [2, 3, 1]
```

```
let cs: Vec<char> = "hello".chars().collect(); // ['h', 'e', ...]  
let t: String = cs.into_iter().collect();      // "hello"
```

```
let m = HashMap::from([('a', 1), ('b', 2)]);  
let p: Vec<(char, i32)> = m.into_iter().collect();
```

- ▶ `into_iter()` hands over the elements one by one, just like `for x in v:` the source is consumed.[†]
- ▶ `collect()` builds **whatever type is asked for**, so the type must be written out: `let s: HashSet<i32>` or `collect::<HashSet<i32>>()`.

[†]A string has no `into_iter()`: it could mean its chars or its bytes, so you have to choose.

Typical beginner errors

- ☹ mixing element types in a `Vec` E0308
- ☹ an empty `Vec::new()` whose element type never turns out E0282
- ☹ indexing past the end panic
- ☹ indexing with an `i32` instead of a `usize` E0277
- ☹ moving an element out by indexing E0507
- ☹ using a collection after `for x in v` E0382
- ☹ modifying a collection while looping over it E0502
- ☹ `collect()` without a target type E0283
- ☹ relying on the order of a `HashMap` or a `HashSet` no error!
- 😊 `get(i)` when the index may be out of range